

SMARTBRIDGE / SKILL WALLET

Code-Layout, Readability and Reusability

Date 30 September 2026
Team ID SWTID-2026-3226
Project Name FitBuddy – AI Fitness Plan Generator using Gemini Models
Maximum Marks 5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the FitBuddy codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Consistent 4-space Python indentation; Jinja templates and CSS are uniformly formatted.
2	Proper File Structure	Files and folders are logically organized	Yes	app/ modules (routes, schemas, models, database, Gemini generators), templates/, static/css/ and tests/ are separated. Minor fix: the package file is named <code>_init_.py</code> and should be <code>__init__.py</code> .
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Descriptive names such as <code>workout_plan</code> , <code>nutrition_tip</code> , <code>experience_level</code> , <code>original_plan</code> and <code>updated_plan</code> .
4	Function / Method Names	Functions are descriptively named	Yes	Functions such as <code>generate_text</code> , <code>generate_workout_gemini</code> , <code>generate_nutrition_tip_with_flash</code> , <code>update_workout_plan</code> and <code>save_generated_plan</code> are clearly named.
5	Code Comments	Inline and block comments explain logic	Partial	Routes have section comments; docstrings and comments can be added to the Gemini prompt and fallback logic.
6	Modular Design	Code is split into reusable functions/modules	Yes	Shared Gemini client, separate workout, nutrition-tip and plan-update generators, plus separate routes, schemas, models and database modules.
7	No Redundant Code	Duplicate or unused code is removed	Partial	Shared <code>gemini_client</code> avoids duplicated API setup; <code>WORKOUT_MODEL</code> is defined in two modules and the HTML and API feedback routes repeat lookup logic, which can be consolidated.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Pydantic validation returns 422, missing user/plan returns 404, and Gemini failures or a missing API key fall back to demo mode.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	<code>gemini_client.py</code>	Python / Google GenAI SDK	Shared Gemini client and <code>generate_text()</code> used by every AI generator; handles API key and empty-response errors	High
2	<code>gemini_generator.py</code>	Python / Google Gemini	7-day workout plan generation; <code>demo_plan()</code> fallback is reused by the plan updater	High
3	<code>gemini_flash_generator.py</code>	Python / Google Gemini Flash	Nutrition and recovery tip generation with goal-based fallback tips	High
4	<code>updated_plan.py</code>	Python / Google Gemini	Revises the workout plan using user feedback in both HTML and API routes	High
5	<code>schemas.py</code>	Python / Pydantic	Input, feedback and response validation shared by HTML form and JSON API routes	High
6	<code>database.py</code> / <code>models.py</code>	Python / SQLAlchemy + SQLite	Database session, User and Plan models used by all routes	Medium

Code-Layout, Readability and Reusability

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clear separation of routes, schemas, models, database and Gemini modules with dedicated templates, static and test folders; <code>_init_.py</code> should be renamed <code>__init__.py</code> .
Readability	4	Descriptive names, type-hinted functions and straightforward request/response flows; docstrings would help further.
Reusability	4	Shared Gemini client and demo fallback avoid duplicated setup; model names are configurable through environment variables; a few duplicated route and constant definitions remain.
Documentation / Comments	3	Section comments exist in routes, but the README only contains the project title; setup instructions and docstrings should be added.
Overall Score	3.75 / 5	Good modular structure with readable code and reusable AI modules; documentation and minor duplication are the main areas to improve.

Implementation Summary:

Area	FitBuddy Implementation
Backend	FastAPI application with a shared APIRouter providing HTML form routes (<code>/generate-workout</code> , <code>/submit-feedback</code> , <code>/view-all-users</code>) and JSON API routes (<code>/api/generate-workout</code> , <code>/api/submit-feedback</code> , <code>/api/users</code>) with Pydantic request validation.
Frontend	Jinja2 templates (<code>base</code> , <code>index</code> , <code>result</code> , <code>all_users</code>) with shared CSS for entering the fitness profile, viewing the 7-day plan, submitting feedback and an admin view.
AI Integration	Google Gemini models are accessed through the google-genai SDK; dedicated modules generate the workout plan, nutrition tip and feedback-based revised plan, with models set by <code>GEMINI_WORKOUT_MODEL</code> and <code>GEMINI_TIP_MODEL</code> .
Data Storage	SQLAlchemy with SQLite (<code>fitbuddy.db</code>) stores users and plans, including the original plan, updated plan, feedback and nutrition tip; <code>DATABASE_URL</code> is configurable.
Fallback Mode	A built-in demo plan and preset nutrition tips are returned when <code>GEMINI_API_KEY</code> is missing or a Gemini call fails.
Testing	pytest with FastAPI TestClient covers the home page, plan generation, feedback update and user listing using a temporary SQLite database in demo mode.
Maintainability	Configuration is environment-based (<code>.env</code> via <code>python-dotenv</code>), dependencies are listed in <code>requirements.txt</code> , and routing, AI, validation and data responsibilities are separated into modules.